GIÁO TRÌNH THỰC HÀNH · LẬP TRÌNH BACKEND

# CHƯƠNG 06

# AUTHENTICATION & AUTHORIZATION

## JWT · MIDDLEWARE · PHÂN QUYỀN

Bài thực hành Buổi 07 — Hệ Liên thông

**NỐI TIẾP PROJECT QUẢN LÝ SINH VIÊN TỪ CHƯƠNG 05**

NODE.JS · EXPRESS · TYPESCRIPT · PNPM · SQL SERVER



TỪ REGISTER / LOGIN ĐẾN BẢO VỆ API THỰC TẾ

TRANG ĐỊNH HƯỚNG

# 01 · Đề bài, mục tiêu và điều kiện chuẩn bị

Tiếp tục bài Chương 05 trên chính project đã có. Dùng AI hỗ trợ từng nhiệm vụ, nhưng sinh viên phải tự build, kiểm thử và giải thích kết quả.

## Đề bài thực hành

Hệ thống **Quản lý sinh viên** hiện có 5 API CRUD, Validation và Swagger, nhưng chưa có lớp bảo vệ theo tài khoản. Hãy nâng cấp hệ thống để **người dùng đăng ký, đăng nhập nhận JWT**; từ đó Backend nhận diện người gửi Request và **chỉ cho Admin thêm, sửa, xóa sinh viên**. Người dùng thường chỉ được xem các API công khai và hồ sơ của chính mình.

- **Đăng ký / đăng nhập:** lưu User trong SQL Server bằng Sequelize, băm mật khẩu với bcrypt; login đúng cấp JWT hết hạn sau 1 giờ.
- **Bảo vệ API:** viết **isAuth** xác minh Bearer Token và **isAdmin** kiểm tra role trước các Route POST/PUT/DELETE SinhVien.
- **Giữ hệ thống cũ:** GET danh sách/chi tiết vẫn public; giữ Zod, Morgan, OpenAPI YAML và hợp đồng response đã học.

## Kết quả cần đạt sau bài thực hành

- **01.** Register hợp lệ trả **201**, trùng email **409**; SQL Server chỉ lưu **passwordHash**, không lưu mật khẩu gốc.
- **02.** Login đúng trả **200 + accessToken**; login sai trả **401**; gọi **/api/auth/profile** chỉ thành công khi JWT hợp lệ.
- **03.** Thao tác ghi không có JWT bị chặn **401**, tài khoản student bị chặn **403**; Admin thao tác CRUD thành công và dữ liệu thay đổi đúng.
- **04.** Project **pnpm build** thành công, có Swagger Bearer Auth, Postman Collection, kết quả **24 ca test** và nhật ký prompt/review.

## Đầu vào bắt buộc

| Thành phần | Điều phải có trước khi bắt đầu |
|---|---|
| Dự án gốc | **backend-quan-ly-sinh-vien**; 5 API CRUD, Middleware Zod/Morgan và file OpenAPI YAML từ Chương 05. |
| Môi trường | Node.js, pnpm, VS Code; có Postman hoặc Swagger UI để kiểm thử. |
| Nguồn dữ liệu | Có **SQL Server + Sequelize** hoạt động. Nếu project còn dùng RAM, báo giảng viên trước khi thực hiện phần User. |
| An toàn | Sao lưu/commit code cũ; chỉ dùng tài khoản và bản ghi thực hành. |

> **Quy trình mỗi bước:** đọc yêu cầu  gửi một prompt  đối chiếu các file thay đổi  **pnpm build**  kiểm thử API thật  lưu bằng chứng. Không coi code AI trả về là kết quả đã đạt.

HỢP ĐỒNG API

# 02 · Phạm vi và API phải giữ đúng

Không xây lại project, không chuyển sang ví dụ Products/Courses và không thay đổi cách lưu bảng SinhVien đang hoạt động.

| Nhóm | Method & endpoint | Quy tắc sau buổi 07 |
|---|---|---|
| Auth | **POST** /api/auth/register | Public; hợp lệ 201, email trùng 409. |
| Auth | **POST** /api/auth/login | Public; đúng 200 + accessToken, sai 401. |
| Auth | **GET** /api/auth/profile | Yêu cầu isAuth; JWT hợp lệ mới trả 200. |
| SinhVien | **GET** /api/v1/sinh-vien<br>**GET** /api/v1/sinh-vien/:maSV | Giữ public và hợp đồng dữ liệu của Chương 04–05. |
| SinhVien | **POST** /api/v1/sinh-vien<br>**PUT** /api/v1/sinh-vien/:maSV<br>**DELETE** /api/v1/sinh-vien/:maSV | Gắn isAuth  isAdmin trước logic cũ; Admin hợp lệ giữ mã 201/200/200. |

## Hai lớp kiểm soát bắt buộc




> **401 Unauthorized:** chưa xác thực hợp lệ (thiếu / sai / hết hạn JWT). **403 Forbidden:** đã xác thực nhưng không đủ vai trò. Đặt isAuth **trước** isAdmin.

**Quy ước response:** giữ **success / message / data** từ các chương trước; trường bổ sung như **accessToken** được mô tả nhất quán trong code, YAML và Postman.

BẢN ĐỒ DỰ ÁN

# 03 · Các file được bổ sung hoặc điều chỉnh

Tên file là gợi ý để thống nhất bài học; nếu dự án có tên file thật khác, giữ cấu trúc hiện có và yêu cầu AI chỉ ra đường dẫn tương ứng.

**CẤU TRÚC THAM CHIẾU**

```
backend-quan-ly-sinh-vien/
|-- package.json                # them package can thiet
|-- .env / .env.example         # JWT_SECRET; KHONG gui .env
|-- sql/04-users-auth.sql       # bang User, khong DROP
|-- src/
|   |-- app.ts / server.ts      # gan auth router mot lan
|   |-- models/user.model.ts    # bang User / Sequelize
|   |-- services/auth.service.ts
|   |-- controllers/auth.controller.ts
|   |-- routes/auth.routes.ts
|   |-- utils/jwt.ts
|   |-- middlewares/isAuth.ts
|   |-- middlewares/isAdmin.ts
|   |-- types/express.d.ts      # neu can khai bao req.user
|   `-- scripts/seed-admin.ts   # chi dung moi truong dev
|-- docs/openapi.yaml           # cap nhat file YAML cu
|-- docs/prompt-log.md          # ghi nhat ky prompt
|-- docs/test-results.md        # bang ket qua thuc te
`-- postman/                    # Collection + test
```

- Giữ nguyên router SinhVien, Controller, Service, Model, DTO và các Validation đã làm; **chỉ ghép Middleware** vào các Route cần bảo vệ.
- Dự án TypeScript ESM/NodeNext đang import nội bộ bằng đuôi **.js** thì tiếp tục đúng quy ước đó.
- Mọi thêm/sửa trên database phải qua script được kiểm tra trước; **không dùng sync({ force: true }), sync({ alter: true })**.

> **Quan trọng:** User dùng để xác thực, không phải bảng SinhVien cũ. Không tự thêm **role, password** vào DTO SinhVien chỉ để làm đăng nhập.

BƯỚC 0 · KIỂM TRA NỀN

# 04 · Khảo sát project trước khi yêu cầu AI code

TERMINAL Ở THƯ MỤC CÓ PACKAGE.JSON

```
node -v
pnpm -v
pnpm install
pnpm build
pnpm dev
```

**01** Gọi **GET /api/v1/sinh-vien** 200; thử lại một ca POST/PUT theo dữ liệu thử và kiểm tra Swagger tại **/api-docs**.

**02** Xác định SQL Server đang kết nối thật qua Sequelize, tên table SinhVien và cấu trúc AppError/response hiện có.

**03** Commit hoặc sao lưu trạng thái chạy đúng; ghi lại PORT, các Middleware đã gắn và vị trí mount router.

**04** Gửi AI các file cần thiết (đã xóa bí mật): package.json, tsconfig, server/app, route, model/service, YAML, .env.example.

**PROMPT 0 · Khảo sát, chưa viết code**

**C** Tôi tiếp tục project quản lý sinh viên sau Chương 05, dùng Express, TypeScript, pnpm, SQL Server + Sequelize; có Zod, Morgan và docs/openapi.yaml.

**T** Đọc các file tôi cung cấp; mô tả luồng Request hiện tại, cách DB kết nối và các vị trí nên thêm register/login, JWT Middleware.

**C** Chưa sửa hay tạo file, không tạo project mới, không đoán nội dung file thiếu, không yêu cầu mật khẩu hoặc toàn bộ .env.

**O** Trả bảng "file / đang làm gì / cần giữ hay sửa / lý do" và danh sách thông tin còn thiếu trước khi triển khai.

**Điểm dừng:** pnpm build và API cũ phải chạy được. Nếu chưa đạt, ghi lỗi hiện tại vào **docs/test-results.md** trước khi sang Bước 1.

BƯỚC 1 · DỮ LIỆU USER

# 05 · Thiết kế bảng tài khoản trong SQL Server

Tài khoản đăng nhập tách khỏi dữ liệu học vụ. Sinh viên tự đăng ký chỉ có vai trò **student**; không được truyền role từ Request để tạo Admin.

| Cột User | Kiểu gợi ý | Ràng buộc |
|---|---|---|
| id | INT IDENTITY | PK, định danh tài khoản. |
| fullName | NVARCHAR(150) | Bắt buộc, không rỗng. |
| email | NVARCHAR(254) | Bắt buộc, UNIQUE; chuẩn hóa nhất quán. |
| passwordHash | NVARCHAR(255) | Lưu hash bcrypt; không lưu plaintext. |
| role | NVARCHAR(20) | Mặc định student; bài này dùng student/admin. |

- Cần có **script SQL không phá dữ liệu**, sau đó Model Sequelize ánh xạ đúng bảng/cột. Nếu bảng dbo.Users đã tồn tại, **khảo sát trước, không tạo lại**.
- Nếu Sequelize cấu hình timestamps, đặt **timestamps: false** khi bảng không có createdAt/updatedAt. Không được tự chèn cột bằng sync alter.

**PROMPT 1 · Tạo schema User và Model**

**C** Backend hiện dùng SQL Server + Sequelize 6 và có bảng SinhVien thật; tôi cần bổ sung riêng bảng dbo.Users để học JWT.

**T** Đề xuất script SQL tạo dbo.Users an toàn nếu chưa có và Model Sequelize gồm id, fullName, email, passwordHash, role; rà soát quyền DB cần thiết.

**C** Không DROP/truncate/sync force/alter; không sửa bảng SinhVien, không lưu mật khẩu rõ; email unique; role mặc định student; giữ ESM/NodeNext.

**O** Liệt kê file, script SQL có điều kiện bảo vệ, Model TypeScript, lệnh chạy trong SSMS và SELECT dùng để xác minh cấu trúc.

**Điểm dừng:** nhìn thấy bảng dbo.Users và các ràng buộc trong SSMS. Chưa cần có tài khoản Admin ngay bước này.

BƯỚC 2 · REGISTER

# 06 · Đăng ký tài khoản và băm mật khẩu

Tạo POST /api/auth/register theo đúng Controller  Service  Sequelize. Dùng Zod/Validation đang có trước khi lưu dữ liệu.

**REQUEST MẪU**

```
POST {{baseUrl}}/api/auth/register
Content-Type: application/json

{
  "fullName": "Nguyen Minh An",
  "email": "an.lab@example.com",
  "password": "SinhVien@123"
}
```

**01** Dữ liệu lỗi định dạng hoặc thiếu trường: **400**; email đã tồn tại: **409**.

**02** Service gọi **bcrypt.hash(password, 12)** và chỉ ghi **passwordHash** vào SQL Server.

**03** Không nhận **role** từ body; backend mặc định **student**.

**04** Đăng ký thành công: **201**, trả thông tin User công khai; tuyệt đối không có password/passwordHash.

**PROMPT 2 · Đăng ký với bcrypt**

**C** Dự án đã có User Model, Zod Middleware và mẫu response success/message/data; cần làm POST /api/auth/register.

**T** Viết Schema, Service, Controller và khai báo Route đăng ký; kiểm tra email trùng, hash bcrypt cost 12 rồi lưu User.

**C** Giữ kiến trúc cũ và SQL Server/Sequelize, không tạo project mới; người đăng ký chỉ là student; từ chối field role; không trả passwordHash.

**O** Trả danh sách file thay đổi, code đầy đủ đúng từng file, các status 201/400/409 và hướng dẫn test cả email trùng.

**Điểm dừng:** POST đúng 201  SELECT bảng dbo.Users thấy **passwordHash** không giống mật khẩu gốc; đăng ký lặp email 409.

BƯỚC 3 · LOGIN

# 07 · Xác thực mật khẩu và cấp Access Token

Client gửi email/password; Service tra cứu SQL Server và kiểm tra bằng bcrypt.compare. Chỉ cấp JWT khi đăng nhập thành công.

**REQUEST / RESPONSE MẪU**

```
POST {{baseUrl}}/api/auth/login
Content-Type: application/json

{
  "email": "an.lab@example.com",
  "password": "SinhVien@123"
}

# Vi du response thanh cong (minh hoa)
{ "success": true, "message": "Login success",
  "data": { "accessToken": "<JWT>", "tokenType": "Bearer",
            "expiresIn": 3600 } }
```

- Email không có hoặc mật khẩu sai đều trả **401** với thông báo chung, không tiết lộ tài khoản nào tồn tại.
- Không viết **user.passwordHash === password**; không chép password/Token vào log của Morgan.
- Payload JWT trong bài dùng **sub** là id của User và **role**; thời hạn mặc định **1h**.

**PROMPT 3 · Login + bcrypt.compare**

**C** Tôi đã hoàn thành Register, lưu passwordHash trong SQL Server và cần đăng nhập bằng email/password.

**T** Viết login Service/Controller: tìm User, dùng bcrypt.compare, đúng thì gọi generateToken và trả accessToken; sai thì 401.

**C** Không lưu plaintext; không log mật khẩu; không phân biệt “email không tồn tại” với “mật khẩu sai”; không dùng jwt.decode để xác minh.

**O** Trả file cần sửa, code TypeScript, ví dụ Request/Response 200 và 401, cùng 3 cách kiểm thử bằng Postman.

BƯỚC 4 · JWT UTILITY

# 08 · Tạo và xác minh JWT đúng cấu hình

Tách thao tác ký/xác minh token vào một module nhỏ. Đây là phần sẽ được isAuth dùng lại cho mọi Request có Bearer Token.

UTILS/JWT.TS - ĐỌC HIỂU

```
// Minh hoa quy uoc cua bai hoc
jwt.sign(
  { sub: String(user.id), role: user.role },
  JWT_SECRET,
  { expiresIn: '1h', algorithm: 'HS256' }
);

jwt.verify(token, JWT_SECRET, {
  algorithms: ['HS256']
});
```

- Cài (nếu thiếu): **pnpm add bcrypt jsonwebtoken** và các gói **@types** nếu TypeScript cần.
- **JWT_SECRET** nằm trong .env; chỉ chia sẻ tên khóa và giá trị giả trong .env.example. App phải báo lỗi cấu hình khi thiếu/secret rỗng.
- JWT đã ký **không mã hóa payload**; không đưa password, passwordHash hoặc thông tin nhạy cảm vào token.

**PROMPT 4 · generateToken và verifyToken**

**C** Project Express + TypeScript ESM, đã có User Model và login; cần module src/utils/jwt.ts dùng jsonwebtoken.

**T** Viết generateToken nhận id/role, ký HS256 hết hạn 1 giờ; viết verifyToken xác minh chữ ký, hạn dùng và kiểu claims.

**C** Bí mật lấy từ process.env.JWT_SECRET; không hard-code; chỉ chấp nhận thuật toán HS256; không tin jwt.decode; giữ import .js của ESM.

**O** Trả code module, hướng dẫn cấu hình .env.example an toàn, vị trí gọi từ login và các trường hợp ký/xác minh thất bại.

**Điểm dừng:** login đúng trả chuỗi JWT có 3 phần (dấu chấm ngăn cách); thay một ký tự trong token rồi xác minh phải **thất bại**.

BƯỚC 5 · AUTHENTICATION

# 09 · Viết Middleware isAuth

Đây là trạm kiểm tra danh tính chung: nhận Authorization: Bearer <token>, xác minh JWT rồi bổ sung thông tin người dùng vào Request.




| Tình huống | Kết quả bắt buộc |
|---|---|
| Không có Authorization, thiếu Bearer, token rỗng | 401; Controller không chạy. |
| Token bị sửa, sai chữ ký hoặc hết hạn | 401; Controller không chạy. |
| Token hợp lệ, claims có kiểu và role được chấp nhận | Gán req.user tối thiểu id/role rồi next(). |

**PROMPT 5 · Tạo isAuth và req.user**

**C** Tôi đã có verifyToken, TypeScript Express và GET /api/auth/profile chưa được bảo vệ.

**T** Viết Middleware isAuth kiểm tra header Bearer, jwt.verify, xác minh payload rồi gán req.user; minh họa khai báo TypeScript cho req.user.

**C** 401 cho thiếu/sai/hết hạn token; không gọi next sau khi gửi lỗi; không trust payload chưa verify; không dùng any hay in secret ra log.

**O** Trả các file cần thêm/sửa và cách test: không token  401, token hợp lệ  req.user; giải thích vì sao phải đặt isAuth trước controller.

**Review TypeScript:** Nếu AI dùng **req.user** nhưng bỏ kiểu dữ liệu, yêu cầu bổ sung khai báo Request rõ ràng thay vì né lỗi bằng **as any**.

BƯỚC 6 · AUTHORIZATION

# 10 · Viết isAdmin, kết hợp hai Middleware

isAuth xác nhận JWT; isAdmin quyết định người đã xác thực có được thao tác quản trị hay không. Đây là hai trách nhiệm khác nhau.

SRC/ROUTES/SINHVIEN.ROUTES.TS - MINH HỌA

```
// Vi du dat dung thu tu
router.delete('/:maSV',
  isAuth,
  isAdmin,
  xoaSinhVien
);
```

| Người gọi | isAuth | isAdmin | Kết quả |
|---|---|---|---|
| Không có JWT | Từ chối | Không chạy | 401 |
| student, JWT hợp lệ | Cho qua | Từ chối | 403 |
| admin, JWT hợp lệ | Cho qua | Cho qua | Vào Controller |

**PROMPT 6 · Tạo isAdmin và giải thích 403**

**C** Middleware isAuth đã gán req.user từ JWT được xác minh; hệ thống học vụ có role student/admin.

**T** Viết isAdmin kiểm tra role là admin, không đủ quyền trả 403; nếu hợp lệ gọi next(); minh họa dùng chung với isAuth.

**C** Không nhận role từ req.body/req.query; không đổi 403 thành 401; nếu thiếu req.user phải xử lý an toàn, không gây lỗi 500.

**O** Cho code src/middlewares/isAdmin.ts, ví dụ gắn vào một Route cũ và 3 ca test thiếu token / student / admin.

**Tự trả lời:** Nếu đưa isAdmin lên trước isAuth, điều gì có thể xảy ra khi req.user chưa được gán?

BƯỚC 7 · GHÉP VÀO API

# 11 · Bảo vệ các Route SinhVien đang có

Không viết lại Controller/Service. Chỉ gắn Middleware trước các handler thao tác ghi dữ liệu; để GET công khai theo phạm vi bài này.

**ROUTE MINH HỌA - GIỮ TÊN HANDLER THẬT**

```
router.get('/', getDanhSachSinhVien);
router.get('/:maSV', getSinhVienTheoMa);

router.post('/', isAuth, isAdmin,
  validate(createSinhVienSchema), taoSinhVien);
router.put('/:maSV', isAuth, isAdmin,
  validate(updateSinhVienSchema), capNhatSinhVien);
router.delete('/:maSV', isAuth, isAdmin, xoaSinhVien);
```

- **Authentication và Authorization** được kiểm tra trước khi chấp nhận thao tác ghi. Validation Zod vẫn bắt dữ liệu sai cho người có quyền.
- Nếu dự án đã có check Role hoặc validation trong Route, **ghép vào chuỗi hiện có**, không tạo router trùng.
- Với Admin đã xác thực, các status cũ 201/200/400/404/409 vẫn phải đúng như trước.

**PROMPT 7 · Tích hợp không làm hỏng CRUD**

**C** Tôi có routes/sinhVien.routes.ts của Chương 05 với năm API CRUD và Zod Validation; nay đã có isAuth/isAdmin.

**T** Đặt isAuth rồi isAdmin vào POST/PUT/DELETE; giữ GET public, giữ nguyên handler, Service, endpoint và quy tắc dữ liệu cũ.

**C** Không tạo router, bảng, database hay app Express thứ hai; không bỏ Zod/Morgan/Swagger; không sửa response thành công của Admin.

**O** Trả diff/file Route sau khi ghép và bảng ca test bảo đảm Admin đúng quyền, student bị chặn, CRUD cũ không hồi quy.

**Điểm dừng:** khi student POST/PUT/DELETE bị 403, phải chứng minh dữ liệu trước/sau không đổi; Admin mới được chạy nghiệp vụ.

BƯỚC 8 · TÀI KHOẢN ADMIN

# 12 · Chuẩn bị tài khoản kiểm thử phân quyền

Để test 403 và đường đi thành công, cần ít nhất hai tài khoản: student đăng ký công khai và admin được tạo từ quy trình nội bộ có kiểm soát.

| Tài khoản | Nguồn tạo | Role | Dùng kiểm thử |
|---|---|---|---|
| student-test | POST /api/auth/register | student | Đăng nhập và thử Route Admin 403. |
| admin-test | Seed script nội bộ trên máy dev | admin | Đăng nhập và gọi thao tác ghi theo hợp đồng CRUD. |

**01** Dùng script seed có kiểm tra email trước khi thêm, bcrypt hash mật khẩu lấy từ input cục bộ và không ghi secret vào repo.

**02** Nếu một tài khoản User đã tồn tại, không tự ý nâng quyền. Không mở API công khai **set-role** hoặc nhận role=admin khi Register.

**03** Sau khi tạo, kiểm tra role trong SQL Server và thử login; không chụp/chia sẻ mật khẩu/token thật.

**PROMPT 8 · Seed Admin an toàn cho môi trường lab**

**C** Tôi cần tài khoản admin-test để kiểm thử RBAC trên môi trường phát triển; User Model đã hoạt động với Sequelize.

**T** Tạo script seed-admin.ts chạy thủ công: kiểm tra email, bcrypt hash password, tạo User role admin; hướng dẫn sử dụng và xác minh.

**C** Không tạo route cấp quyền public; không đưa password mặc định vào source/SQL hay commit; không cập nhật role người dùng hiện có nếu chưa cho phép.

**O** Cho file seed, lệnh chạy tương thích scripts/tsx hiện có, cách xác nhận dữ liệu trong SSMS, cách xóa dữ liệu thử an toàn.

**Chú ý:** Tài khoản kết nối ứng dụng phải có quyền tối thiểu phù hợp. Nếu bị từ chối CREATE/INSERT, kiểm tra quyền với giảng viên; không tự dùng sa hoặc tắt bảo vệ DB.

BƯỚC 9 · SWAGGER YAML

# 13 · Mô tả Bearer Auth trong OpenAPI 3.0

Tái sử dụng **docs/openapi.yaml** của Chương 04–05, không đổi sang JSDoc @swagger và không tạo file đặc tả thứ hai.

**ĐOẠN YAML MINH HỌA - GHÉP VÀO YAML HIỆN CÓ**

```
components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT

# Security on profile and POST/PUT/DELETE
security:
  - bearerAuth: []
```

- Khai báo **/api/auth/register** (201/400/409), **/api/auth/login** (200/400/401), **/api/auth/profile** (200/401).
- Mô tả 401 ở các Route cần JWT, 403 ở Route yêu cầu Admin; giữ các response 400/404/409 vốn có.
- Trong Swagger Authorize, nhập token theo hướng dẫn của giao diện Bearer; kiểm thử phải nhìn **Request thực tế** có header Authorization.

**PROMPT 9 · Bổ sung OpenAPI đúng chuẩn**

**C** Project đã có docs/openapi.yaml OpenAPI 3.0 và Swagger UI tại /api-docs; các API SinhVien đã được mô tả.

**T** Thêm register/login/profile, components.securitySchemes.bearerAuth và security cho các Route được bảo vệ; cập nhật response 401/403.

**C** Không tạo tài liệu Swagger khác, không tạo key components trùng, không thay schemas SinhVien hoặc mẫu response cũ.

**O** Trả diff YAML đúng vị trí và checklist mở /api-docs: Authorize, thử 401/403 và Login trả accessToken.

**Điểm dừng:** YAML mở được, bảo mật xuất hiện ở đúng endpoint và Swagger "Try it out" gửi request thật tới Backend.

BƯỚC 10 · POSTMAN

# 14 · Chuẩn bị Collection và token kiểm thử

Dùng lại Collection của các buổi trước. Tạo Environment để không nhập lặp baseUrl và dễ đổi giữa student/admin khi test.

| Biến | Giá trị minh họa / cách lấy | Lưu ý |
|---|---|---|
| baseUrl | http://localhost:5000 | Đổi đúng PORT đang chạy. |
| studentToken | Từ login bằng tài khoản student-test | Không đưa giá trị thật vào bài nộp. |
| adminToken | Từ login bằng tài khoản admin-test | Không đưa token thật lên GitHub. |
| testMaSV | SV098 (nếu chưa tồn tại) | Phải khớp quy tắc mã SV hiện có. |

HEADER THỬ NGHIỆM

```
GET {{baseUrl}}/api/auth/profile
Authorization: Bearer {{studentToken}}

DELETE {{baseUrl}}/api/v1/sinh-vien/{{testMaSV}}
Authorization: Bearer {{adminToken}}
```

**01** Tạo 3 Request Auth: Register, Login, Profile; lưu cùng Collection CRUD SinhVien.

**02** Lấy token bằng POST Login; lưu vào biến Postman **chỉ trên máy kiểm thử**. Trước mỗi test, xem biến đang trỏ đúng loại tài khoản.

**03** Thực thi các ca theo bảng sau, ghi Status thực tế, nội dung lỗi và minh chứng trước/sau đối với Request ghi dữ liệu.

**PROMPT 10 · Hướng dẫn bộ kiểm thử Postman**

**C** Tôi đã có register/login/profile, isAuth/isAdmin và CRUD SinhVien. Tôi kiểm thử bằng Postman Environment.

**T** Viết trình tự lấy studentToken/adminToken và tạo Request có Bearer Authorization; mô tả cách assert status code và response.

**C** Không tạo token giả hoặc tự cho qua; không tự kết luận PASS; không để token/mật khẩu thật trong Collection export.

**O** Trả danh sách request theo thứ tự, headers/body, expected status và mẫu Postman Tests dùng được.

BƯỚC 10 · MA TRẬN KIỂM THỬ

# 15 · Bảng test Register, Login và JWT

Chạy theo thứ tự. Tài khoản email test dùng địa chỉ giả trong database bài lab, không dùng tài khoản thật.

| Mã | Request / tình huống | Mong đợi | Bằng chứng kiểm chứng |
|---|---|---|---|
| A01 | Register email mới, body hợp lệ | 201 | User tồn tại; role=student. |
| A02 | Register trùng email A01 | 409 | Không tăng số User. |
| A03 | Register body thêm role=admin | 400 | Không tạo tài khoản Admin. |
| A04 | Register thiếu password/email sai | 400 | Không lưu dữ liệu lỗi. |
| A05 | Login đúng tài khoản student | 200 | data.accessToken hợp lệ. |
| A06 | Login sai password | 401 | Không trả accessToken. |
| A07 | Login email không tồn tại | 401 | Không lộ thông tin tồn tại User. |
| A08 | GET /api/auth/profile không có token | 401 | Không nhận data hồ sơ. |
| A09 | Profile với token bị sửa 1 ký tự | 401 | Chữ ký không qua xác minh. |
| A10 | Profile với JWT student hợp lệ | 200 | Hồ sơ đúng user, không hash. |
| A11 | Token hết hạn  gọi Profile | 401 | Lỗi hết hạn token. |
| A12 | Register/login và xem SQL Server | PASS | Không plaintext password, không log JWT. |

> **Test A11:** Tạo token sống rất ngắn **chỉ trong môi trường test** hoặc dùng test tự động có điều khiển thời gian. Không thay cấu hình JWT 1 giờ của bài chính rồi quên đổi lại.

**Lưu trong docs/test-results.md:** Mã ca test | Request | Kết quả mong đợi | Status thực tế | Bằng chứng | PASS/FAIL | Nguyên nhân nếu FAIL.

BƯỚC 10 · MA TRẬN PHÂN QUYỀN

# 16 · Bảng test isAuth và isAdmin trên SinhVien

Các ca ghi dữ liệu phải dùng mã sinh viên thử không trùng và không có quan hệ khóa ngoại gây xóa nhầm. Với SQL Server, luôn kiểm tra lại bằng GET/SELECT.

| Mã | Request / tài khoản | Mong đợi | Kiểm chứng thêm |
|---|---|---|---|
| B01 | GET danh sách, không JWT | 200 | data vẫn là mảng. |
| B02 | POST SinhVien, không JWT | 401 | Không INSERT bản ghi. |
| B03 | PUT SinhVien, token không hợp lệ | 401 | Giá trị cũ không đổi. |
| B04 | DELETE SinhVien, JWT student | 403 | SinhVien vẫn tồn tại. |
| B05 | POST SinhVien, JWT student | 403 | Không INSERT. |
| B06 | PUT SinhVien, JWT student | 403 | Không UPDATE. |
| B07 | POST SinhVien hợp lệ, JWT admin | 201 | GET lại đúng bản ghi mới. |
| B08 | PUT SinhVien hợp lệ, JWT admin | 200 | GET lại thấy thay đổi. |
| B09 | DELETE SinhVien, JWT admin | 200 | GET lại  404. |
| B10 | Admin POST body sai Zod | 400 | Trường errors chỉ ra lỗi. |
| B11 | Admin POST mã bị trùng | 409 | Không tạo bản ghi trùng. |
| B12 | DELETE cùng mã lần thứ 2 bằng Admin | 404 | Không còn dữ liệu để xóa. |

> **Chứng minh Middleware chặn thật:** Chụp response 401/403 và GET/SELECT sau đó. Chỉ nhìn Swagger có biểu tượng ổ khóa **không chứng minh** Backend đã phân quyền.

**Kiểm thử hồi quy:** lặp các test Chương 05 về Zod, Morgan và các status cũ với tài khoản Admin ở Route thao tác ghi; không tự sửa Expected cho khớp lỗi đang gặp.

BƯỚC 11 · SECURITY REVIEW

# 17 · Phát hiện lỗi từ code AI sinh ra

Đọc kỹ đoạn mã **cố tình sai** sau. Không chạy hoặc ghép vào project. Hãy mô tả rủi ro và yêu cầu AI đề xuất bản sửa theo nguyên tắc đã học.

KHÔNG SAO CHÉP VÀO DỰ ÁN

```
// DOAN CODE SAI - CHI DE REVIEW
if (user.password === password) {
  const token = jwt.sign(
    { id: user.id, role: req.body.role },
    '123456'
  );
  return res.json({ token, user });
}

const payload = jwt.decode(req.headers.authorization);
if (payload.role === 'admin') next();
```

| Dấu hiệu cần tìm | Câu hỏi review |
|---|---|
| Password plaintext | Có dùng bcrypt.hash / bcrypt.compare không? |
| JWT_SECRET hard-code và yếu | Secret có nằm trong .env mạnh và không lộ trên repo? |
| Thiếu expiresIn | Token có thời hạn 1 giờ không? |
| Tin role từ Request | Role có lấy từ User đã xác thực không? |
| jwt.decode thay cho jwt.verify | Chữ ký, hạn và thuật toán có được kiểm tra không? |
| Trả nguyên object user | Có vô tình trả passwordHash ra Client? |

**PROMPT 11 · Đóng vai Security Reviewer**

**C** Đây là code Register/Login/isAuth/isAdmin của project học tập; tôi cần review bảo mật trước khi nộp.

**T** Chỉ ra từng lỗ hổng về hash password, secret, hạn JWT, xác minh chữ ký, role do Client gửi, dữ liệu nhạy cảm trong response.

**C** Phân biệt 401/403; ưu tiên thay đổi nhỏ; không đề xuất tắt xác thực hoặc thay toàn bộ stack; không cần secret thật.

**O** Xuất bảng “file/dòng - nguy cơ - cách sửa - test chứng minh”, sau đó đưa bản vá cần thiết.

BƯỚC 12 · XỬ LÝ LỖI

# 18 · Khi AI viết code nhưng chương trình chưa chạy

Không đổi cấu hình hàng loạt để làm mất lỗi. Lấy bằng chứng đầy đủ, yêu cầu sửa đúng file và phải chạy lại test từng thất bại.

| Triệu chứng | Điều cần kiểm tra trước |
|---|---|
| 401 dù đã Login | Header có Bearer và khoảng trắng đúng? Token lấy từ lần login mới nhất? Secret khi ký và verify giống nhau? |
| 403 với Admin | Admin seed có role=admin? Login lại lấy token mới sau khi thay Role? isAuth đã gán req.user? |
| TypeScript báo req.user không có | Khai báo mở rộng Request có được TypeScript nhận diện? Có dùng đúng kiểu payload? |
| 500 khi Register | DB/table dbo.Users/permission/Sequelize mapping; cột tên đúng không? Có lỗi unique được ánh xạ 409? |
| Swagger 404 | Route đã mount vào app cũ chưa? Đường dẫn trên YAML có trùng với URL runtime? |
| API cũ hỏng sau khi ghép | So sánh diff file Route; kiểm tra thứ tự isAuth, isAdmin, validate và handlers. |

**PROMPT SỬA LỖI · Chỉ sửa nguyên nhân nhỏ nhất**

**C** Tôi đang làm Bước [số] trong backend-quan-ly-sinh-vien (Express + TypeScript + Sequelize + JWT).

**T** Phân tích lỗi xảy ra khi [method + URL + thao tác]; lỗi đầy đủ: [dán terminal/status/response]; file hiện tại: [đính kèm đã bỏ secret].

**C** Giữ pnpm, ESM/NodeNext, SQL Server, Zod, Morgan, YAML và CRUD; không tạo app mới, không bỏ kiểm tra quyền để chữa lỗi.

**O** Giải thích nguyên nhân, chỉ ra diff sửa nhỏ nhất trong đúng file và test tái hiện + regression test sau khi sửa.

**Mẫu một dòng prompt-log:** Bước | Prompt đã sửa thế nào | AI đề xuất file nào | Tôi chấp nhận/từ chối gì | Bằng chứng build/test | Điều tôi giải thích được.

TỔNG KẾT · NỘP BÀI

# 19 · Hồ sơ nộp và thang đánh giá

## Sinh viên nộp các thành phần sau

**01 Mã nguồn project gốc đã nâng cấp:** Route/Auth Service/Model/Middleware/Utility/Seed nếu có. Không nộp node_modules, dist hoặc .env.

**02 SQL:** script tạo bảng dbo.Users không phá dữ liệu, ảnh SSMS thể hiện User đã đăng ký có passwordHash (che dữ liệu nhạy cảm).

**03 OpenAPI:** docs/openapi.yaml và ảnh Swagger thể hiện Bearer Auth, Register/Login/Profile, response 401/403.

**04 Postman:** Collection (đã xóa giá trị token thật) cùng kết quả test A01–A12 và B01–B12.

**05 docs/prompt-log.md** và **docs/test-results.md** có Expected/Actual, ảnh chứng minh ít nhất 401, 403, 200, 201, 409.

**06 Giải thích cá nhân:** 5–8 câu ngắn về bcrypt, JWT payload, isAuth, isAdmin, next() và phân biệt 401/403.

## Gợi ý rubric chấm 10 điểm

| Tiêu chí | Điểm | Minh chứng tối thiểu |
|---|---|---|
| Register/Login + SQL Server + bcrypt | 3.0 | 201/409/200/401, DB hash, không lộ hash. |
| JWT và Middleware isAuth/isAdmin | 3.0 | 401 thiếu/sai token, 403 student, admin qua. |
| Tích hợp đúng API + Swagger + Postman | 2.0 | CRUD không hồi quy, ca A/B có bằng chứng. |
| Prompt, code review, giải thích | 2.0 | Prompt log, nhận diện lỗ hổng và cách sửa. |

> **Không đạt yêu cầu cốt lõi:** hard-code JWT_SECRET; lưu mật khẩu dạng rõ; cho phép Client tự tạo admin; dùng jwt.decode để xét quyền; chỉ sửa Swagger mà API vẫn không được Middleware bảo vệ.

PHẦN TỰ KIỂM TRA

# 20 · Câu hỏi bảo vệ bài thực hành

Trả lời bằng lời của mình, không dùng lại nguyên đoạn mô tả do AI sinh ra. Có thể minh họa bằng chính file đã sửa trong project.

**01** Vì sao Register dùng **bcrypt.hash()** nhưng Login phải dùng **bcrypt.compare()**?

**02** Vì sao JWT có chữ ký số mà vẫn **không nên chứa passwordHash** trong payload?

**03** Nếu User sửa chữ **student** thành **admin** trong payload JWT, điều gì xảy ra khi isAuth gọi jwt.verify?

**04** Phân biệt **401** và **403** bằng hai Request cụ thể trong B04 và A08.

**05** Tại sao phải gắn **isAuth trước isAdmin**? req.user được tạo ở đâu?

**06** Nếu Route DELETE chỉ được đánh dấu security trong Swagger nhưng không gắn isAdmin ở Express, có an toàn không?

**07** Nếu thay role của một User trong SQL Server sau khi đã cấp JWT, token cũ có cập nhật role ngay không? Muốn kiểm soát phải làm gì?

**08** Tại sao test 403 cần gọi GET/SELECT lại và không thể chỉ nhìn status là đủ?

## Nhìn sang các buổi tiếp theo

- **Buổi 08 - Multer:** bảo vệ endpoint upload avatar bằng isAuth và gắn avatar cho chính User đang đăng nhập; phần upload/chặn file 2 MB sẽ học ở buổi sau.
- **Buổi 09 - Vite/Axios:** giao diện đăng nhập lấy accessToken rồi gọi API có header Bearer, cấu hình CORS và xử lý 401/403.

> **Hoàn thành bài học:** Không chỉ "API login trả về token", mà phải chứng minh được **JWT hợp lệ đi qua, JWT sai bị chặn, student thiếu quyền bị từ chối và Admin thao tác đúng**.